SpatFD v2: diseño de la predicción y funcionamiento del *solver*

Documento de diseño

Octubre de 2026

Este documento describe lo que se diseñó para la predicción en SpatFD v2. El centro es el *solver* de kriging: qué sistema resuelve, cómo lo resuelve y cómo comprobar a mano que sus soluciones son correctas. La última parte plantea cómo se integraría un método `lambda` multivariado, que todavía **no está implementado**.

Índice

1. Panorama del diseño **1**
- 1.1. El objeto y el registro de modelos . . . 1
- 1.2. Las capas de la predicción . . . 1

2. De variogramas a covarianzas **2**

3. El *solver* **2**
- 3.1. El problema . . . 2
- 3.2. La matriz por bloques . . . 2
- 3.3. El algoritmo . . . 3
- 3.4. De los *scores* a las curvas . . . 3

4. Cómo verificar las soluciones **3**
- 4.1. A mano, con un solo *score* . . . 3
- 4.2. Propiedades que toda solución debe cumplir . . . 4
- 4.3. Comparaciones externas . . . 4

5. El método `lambda` univariado **5**
- 5.1. El sistema . . . 5
- 5.2. Lo que se implementó . . . 5
- 5.3. Cómo verificarlo . . . 5

6. Integración de `lambda` multivariado **5**
- 6.1. El sistema . . . 6
- 6.2. Cómo encaja en el *solver* . . . 6
- 6.3. Lo que hace falta . . . 6
- 6.4. Cómo se verificaría . . . 7

7. Limitaciones del *solver* **7**

1. Panorama del diseño

1.1. El objeto y el registro de modelos

Un objeto `SpatFD` guarda los *scores* de los componentes principales de cada variable y, como atributos, lo necesario para volver de los *scores* a las curvas. Los modelos de variograma se

registran dentro del mismo objeto:

- `add_model()` traduce el modelo (si viene de `gstat`), lo compila y lo guarda con un nombre. La compilación convierte cada variograma en una función de la distancia, $\gamma(h)$, y se hace una sola vez.
- Cada modelo registrado guarda tres cosas: la tabla original, los variogramas compilados y la meseta total de cada uno (la suma de sus mesetas parciales).
- `sfd$models` es la tabla resumen de lo registrado. Se calcula al consultarla.

1.2. Las capas de la predicción

`predict()` es deliberadamente delgada. El trabajo está repartido así:

Función	Qué hace
`prediction_setup()`	Verifica argumentos y prepara sitios, valores y modelo
`score_covariances()`	Convierte los variogramas en funciones de covarianza
`simple_kriging()`	El *solver*: predice los *scores* y su varianza
`krige_scores()`	Une las tres anteriores
`predict_scores()`	Reconstruye las curvas a partir de los *scores*
`predict_lambda()`	Método `lambda`, con su propio sistema
`predict()`	Escoge el método y arma el resultado

Los dos métodos devuelven lo mismo: para cada variable, las curvas predichas (`$pred`) y su varianza de predicción (`$var`). Los gráficos, los mapas y la validación cruzada (`leave_one_out()`) solo dependen de esa salida, no del método.

2. De variogramas a covarianzas

El kriging simple trabaja con covarianzas. De cada variograma compilado se obtiene

$$C(h) = C(0) - \gamma(h), \qquad C(0) = \text{suma de las mesetas parciales.}$$

Tres detalles importan para verificar resultados:

- El efecto pepita vale cero en $h = 0$, de modo que $C(0)$ incluye la pepita y $C(h)$ para $h > 0$ no. Por eso la predicción en un sitio observado devuelve el valor observado.
- Para un variograma cruzado, $C(0)$ es la suma de sus mesetas parciales, que pueden ser negativas. Se supone covarianza cruzada simétrica: $C_{ij}(h) = C_{ji}(h)$.
- Los modelos sin meseta no tienen covarianza y se rechazan.

3. El *solver*

3.1. El problema

Un *campo* es un *score*, por ejemplo `PM10.PC1`. Hay q campos; el campo j se observó en n_j sitios con valores z_j. Se apilan todas las observaciones en un vector

$$z = \begin{bmatrix} z_1 \\ \vdots \\ z_q \end{bmatrix} \in \mathbb{R}^N, \qquad N = n_1 + \cdots + n_q.$$

El predictor de kriging simple con media cero para el campo j en un sitio nuevo s_0 y su varianza son

$$\hat{f}_j(s_0) = c_{0j}^T \, \Sigma^{-1} z, \qquad \sigma_j^2(s_0) = C_{jj}(0) - c_{0j}^T \, \Sigma^{-1} c_{0j}. \tag{1}$$

3.2. La matriz por bloques

Σ es la covarianza entre todas las observaciones. Se arma por bloques, uno por cada pareja de campos: el bloque (i, j) tiene tamaño $n_i \times n_j$ y es la covarianza C_{ij} evaluada en las distancias entre los sitios del campo i y los del campo j. Con dos *scores* de una variable a y uno de una variable b:

$$\Sigma = \begin{bmatrix} \Sigma_{a_1a_1} & \Sigma_{a_1a_2} & \Sigma_{a_1b_1} \\ \Sigma_{a_2a_1} & \Sigma_{a_2a_2} & \Sigma_{a_2b_1} \\ \Sigma_{b_1a_1} & \Sigma_{b_1a_2} & \Sigma_{b_1b_1} \end{bmatrix}, \qquad c_{0,a_1} = \begin{bmatrix} C_{a_1a_1}(d_{a,0}) \\ C_{a_2a_1}(d_{a,0}) \\ C_{b_1a_1}(d_{b,0}) \end{bmatrix},$$

donde $d_{a,0}$ son las distancias de los sitios de a al sitio nuevo.

Si el modelo no tiene variograma cruzado para una pareja, su bloque es cero. De ahí sale la unificación de los dos casos:

- **Sin variogramas cruzados**, Σ es diagonal por bloques y el sistema se separa en un kriging independiente por *score*.
- **Con variogramas cruzados**, es el sistema de cokriging simple.

Los sitios no tienen que coincidir entre variables: cada bloque usa los sitios de sus dos campos.

3.3. El algoritmo

1. **Distancias.** Los *scores* de una misma variable comparten sitios, así que cada matriz de distancias se calcula una vez por conjunto distinto de sitios. Las distancias se calculan coordenada a coordenada para que la de un sitio consigo mismo sea exactamente cero, de lo que depende el efecto pepita.

2. **Grupos.** Dos campos pertenecen al mismo grupo si están conectados por covarianzas cruzadas, directamente o a través de otros campos. Los grupos no comparten información y se resuelven por aparte; el resultado es idéntico al del sistema completo.

3. **Armado.** Para cada grupo se construyen Σ y la matriz C_0, cuyas columnas son los c_{0j} de todos los sitios nuevos y todos los campos. Como Σ es simétrica, solo se evalúa un bloque de cada pareja y el otro es su traspuesta.

4. **Factorización.** Se calcula la factorización de Cholesky $\Sigma = R^T R$, con R triangular superior, una sola vez por grupo. Si falla, Σ no es definida positiva (sitios duplicados o modelo inválido) y la función se detiene con ese mensaje.

5. **Solución.** Se resuelven dos sistemas triangulares,

   $$V = R^{-T} C_0, \qquad w = R^{-T} z,$$

   y con ellos

   $$\hat{f} = V^T w, \qquad \sigma^2 = C(0) - (\text{suma por columnas de } V^2). \tag{2}$$

Por qué (2) equivale a (1). Como $\Sigma^{-1} = R^{-1}R^{-T}$,

$$C_0^T \Sigma^{-1} z = (R^{-T}C_0)^T (R^{-T} z) = V^T w, \qquad c_0^T \Sigma^{-1} c_0 = \|R^{-T} c_0\|^2,$$

y esta última es la suma de los cuadrados de la columna correspondiente de V. Los pesos de kriging $\Sigma^{-1}c_0$ no se calculan explícitamente: basta una resolución triangular en lugar de dos.

3.4. De los *scores* a las curvas

Con los *scores* predichos de una variable, la curva es

$$\hat{\chi}_{s_0}(t) = \mu(t) + \sum_{k=1}^{K} \hat{f}_k(s_0)\, \xi_k(t),$$

y se calcula sobre los coeficientes de la base: los coeficientes de la media más los de las funciones propias multiplicados por los *scores*. La varianza reportada es $\sum_k \sigma_k^2(s_0)$.

4. Cómo verificar las soluciones

4.1. A mano, con un solo *score*

El siguiente código resuelve el sistema directamente con `solve()` y lo compara con el *solver*. Usa los datos de México y un modelo fijo.

```
library(SpatFD)
data(air_mexico)

pm10 <- spatfd(pm10_mexico, coords = as.matrix(coords_pm10_mexico),
               basis = "fourier", nbasis = 21, lambda = 0.000001,
               nharm = 2, name = "PM10")
add_model(pm10, list(
  PM10.PC1 = gstat::vgm(200000, "Exp", 8000, nugget = 20000),
  PM10.PC2 = gstat::vgm(60000,  "Sph", 15000)), name = "m")
nuevo <- cbind(X = 490000, Y = 2150000)

# --- A mano ---
modelo <- attr(pm10, "models")[["m"]]
sitios <- pm10$scores[, c("X", "Y")]
z      <- pm10$scores[, "PM10.PC1"]

covarianza <- function(h)
  modelo$sills[["PM10.PC1"]] - modelo$variograms$PM10.PC1(h)

Sigma <- covarianza(as.matrix(dist(sitios)))
c0    <- covarianza(sqrt(colSums((t(sitios) - as.vector(nuevo))^2)))

pesos <- solve(Sigma, c0)
c(prediccion = sum(pesos * z),
  varianza   = covarianza(0) - sum(pesos * c0))

# --- Con el paquete ---
SpatFD:::krige_scores(pm10, nuevo, model = "m")
```

Los dos caminos dan lo mismo: predicción 273,0496 y varianza 132355,0234 para `PM10.PC1`.

4.2. Propiedades que toda solución debe cumplir

- **Interpolación exacta.** Al predecir en un sitio observado se recupera el *score* observado y la varianza es cero. En el ejemplo, la diferencia es del orden de 10^{-14} y la varianza de 10^{-11}.
- **Varianza acotada.** Con el método `scores`, $0 \le \sigma_j^2(s_0) \le C_{jj}(0)$. Lejos de todos los sitios la varianza tiende a la meseta y la predicción a cero, es decir, la curva tiende a la media.

- **Bloques independientes.** Con un modelo sin variogramas cruzados, predecir varias variables juntas da lo mismo que predecir cada una por aparte.
- **Matriz definida positiva.** Los valores propios de Σ deben ser positivos. En el ejemplo el menor es 91182,6.
- **Simetría.** Intercambiar el orden de las variables no cambia el resultado.

4.3. Comparaciones externas

- **Contra gstat.** El script `dev/02_comparacion_gstat.R` compara variogramas, kriging, cokriging y `lambda`. Las diferencias relativas están entre 10^{-16} y 10^{-13}.
- **Contra la versión CRAN.** Las curvas y varianzas de los métodos `scores` y `lambda` coinciden con las de `KS_scores_lambdas()` con diferencias del orden de 10^{-16}.
- **Pruebas automáticas.** Los archivos de `tests/testthat` repiten estas comparaciones con datos simulados, incluido un caso de cokriging con sitios distintos por variable, pepita y mesetas cruzadas negativas.

Una diferencia mayor que 10^{-8} en cualquiera de estas comparaciones indica un error, no redondeo.

5. El método `lambda` univariado

5.1. El sistema

La curva se predice como combinación lineal de las curvas observadas, con un peso escalar por sitio:

$$\hat{\chi}_{s_0}(t) = \sum_{i=1}^{n} \lambda_i \, \chi_{s_i}(t).$$

Los pesos minimizan $E\|\chi_{s_0} - \hat{\chi}_{s_0}\|^2$. Si las curvas se escriben con sus componentes principales, el producto interno entre dos curvas centradas es

$$E\langle \chi_{s_i}, \chi_{s_j} \rangle = \sum_k \sum_l \mathrm{Cov}\big(f_k(s_i), f_l(s_j)\big) \langle \xi_k, \xi_l \rangle = \sum_k C_{kk}(s_i - s_j),$$

porque las funciones propias son ortonormales. Por eso el sistema es

$$\Omega \, \lambda = c, \qquad \Omega = \sum_{k=1}^{K} \Omega_k, \quad c = \sum_{k=1}^{K} c_k,$$

con Ω_k y c_k las matrices de covarianza del *score* k. Una consecuencia: los variogramas cruzados entre *scores* de la misma variable no intervienen en `lambda`, porque $\langle \xi_k, \xi_l \rangle = 0$ para $k \neq l$.

5.2. Lo que se implementó

Por decisión explícita, igual que la versión CRAN:

- Los pesos se aplican a las curvas *sin centrar*. En el ejemplo de la sección anterior los pesos suman 0,844, no uno, así que la curva predicha queda encogida hacia cero.
- La varianza es $\sum_k \big(\eta_k - 2\, c_k^T \lambda + \lambda^T \Omega_k \lambda\big)$, con η_k el valor propio del componente y no la meseta del modelo. Puede salir negativa.
- Solo una variable y modelos sin variogramas cruzados.

5.3. Cómo verificarlo

Los pesos y la curva se reproducen a mano, continuando el código anterior:

```
cov2  <- function(h)
  modelo$sills[["PM10.PC2"]] - modelo$variograms$PM10.PC2(h)
D     <- as.matrix(dist(sitios))
d0    <- sqrt(colSums((t(sitios) - as.vector(nuevo))^2))

lambda <- solve(covarianza(D) + cov2(D), covarianza(d0) + cov2(d0))
curva  <- attr(pm10, "fd")$PM10$coefs[, rownames(sitios)] %*% lambda

pred <- predict(pm10, nuevo, model = "m", method = "lambda")
max(abs(curva - pred$PM10$pred$coefs))      # del orden de 1e-13
```

6. Integración de `lambda` multivariado

Estado: no implementado. Lo que sigue es una propuesta. Es la extensión directa del criterio univariado y no existe en la versión CRAN, de modo que no hay una implementación de referencia contra la cual compararla. Conviene contrastarla con la formulación de Bohórquez, Giraldo y Mateu (2017) antes de programarla.

6.1. El sistema

Con dos variables a y b, el predictor de la curva de a usaría también las curvas de b:

$$\hat{\chi}^a_{s_0}(t) = \mu^a(t) + \sum_{i=1}^{n_a} \lambda_i \big(\chi^a_{s_i}(t) - \mu^a(t)\big) + \sum_{j=1}^{n_b} \nu_j \big(\chi^b_{s_j}(t) - \mu^b(t)\big).$$

Minimizar $E\|\chi^a_{s_0} - \hat{\chi}^a_{s_0}\|^2$ lleva a un sistema por bloques, ahora con un bloque por *variable*:

$$\begin{bmatrix} \Omega_{aa} & \Omega_{ab} \\ \Omega_{ba} & \Omega_{bb} \end{bmatrix} \begin{bmatrix} \lambda \\ \nu \end{bmatrix} = \begin{bmatrix} c_a \\ c_b \end{bmatrix}. \tag{3}$$

Los bloques de una misma variable son las sumas ya conocidas,

$$\Omega_{aa}(h) = \sum_k C^{aa}_{kk}(h), \qquad \Omega_{bb}(h) = \sum_l C^{bb}_{ll}(h),$$

pero el bloque cruzado necesita los productos internos entre las funciones propias de las dos variables, que ya no son ortogonales entre sí:

$$\Omega_{ab}(h) = \sum_k \sum_l G_{kl}\, C^{ab}_{kl}(h), \qquad G_{kl} = \langle \xi^a_k, \xi^b_l \rangle. \tag{4}$$

Los lados derechos son $c_a = \Omega_{aa}(d_{a,0})$ y $c_b = \Omega_{ba}(d_{b,0})$, y la varianza de predicción sería

$$\Omega_{aa}(0) - \begin{bmatrix} c_a \\ c_b \end{bmatrix}^T \begin{bmatrix} \Omega_{aa} & \Omega_{ab} \\ \Omega_{ba} & \Omega_{bb} \end{bmatrix}^{-1} \begin{bmatrix} c_a \\ c_b \end{bmatrix}.$$

6.2. Cómo encaja en el *solver*

El sistema (3) tiene exactamente la forma que el *solver* ya resuelve. Lo que cambia es qué son los campos, las covarianzas y los valores:

	Método `scores`	Método `lambda`
Campo	un *score*	una variable
Covarianza	$C_{ij}(h)$ del modelo	$\Omega_{ab}(h)$ agregada, ecuación (4)
Valores	un número por sitio	una curva por sitio (sus coeficientes)
Salida	*scores* predichos	coeficientes de la curva predicha

El único cambio necesario en `simple_kriging()` es aceptar que los valores de cada campo sean una matriz (una fila por sitio, una columna por coeficiente de la base) en lugar de un vector. Las fórmulas (2) valen sin cambio: $w = R^{-T}Z$ pasa a ser una matriz y $V^T w$ da directamente los coeficientes de las curvas predichas.

Con ese cambio, el método univariado actual se podría reescribir sobre el mismo *solver* (un solo campo con covarianza $\sum_k C_{kk}$), lo que da una prueba inmediata: debe reproducir los resultados de hoy.

6.3. Lo que hace falta

1. **Un dominio y una base comunes.** Para sumar curvas de b a la predicción de a, ambas deben estar definidas sobre el mismo intervalo de tiempo y expresadas en la misma base. Hoy no se exige: en los datos de México, PM10 tiene 4344 instantes y NO2 tiene 4292. Habría que exigirlo para este método o proyectar las curvas de una variable en la base de la otra.

2. **La matriz G.** Con una base común de matriz de productos internos J, y con Ξ^a, Ξ^b las matrices de coeficientes de las funciones propias, $G = (\Xi^a)^T J\, \Xi^b$.

3. **Las covarianzas agregadas.** Una función por pareja de variables, construida a partir de los variogramas compilados y de G. Para las parejas de *scores* sin variograma cruzado el término se omite.

4. **El *solver* con valores matriciales**, como se describió.

5. **Dos decisiones de criterio.**

   - Centrar las curvas. Con varias variables no parece defendible sumar curvas sin centrar de una variable distinta; esto obligaría a decidir también qué hacer con el caso univariado, que hoy sigue a CRAN.
   - Usar la meseta del modelo o el valor propio en la varianza.

6.4. Cómo se verificaría

Como no hay referencia externa, la verificación tendría que apoyarse en propiedades:

- Sin variogramas cruzados entre variables, o con $G = 0$, debe dar exactamente el `lambda` univariado de cada variable.
- Debe coincidir con la solución directa de (3) mediante `solve()`, como en las verificaciones de este documento.
- Con curvas centradas, predecir en un sitio observado debe devolver la curva observada.
- La validación cruzada debe poder compararlo con el cokriging por *scores* sobre el mismo modelo.

7. Limitaciones del *solver*

- **Vecindario global.** Usa todos los sitios a la vez. El costo crece con el cubo del número de observaciones de cada grupo.
- **Sin memoria entre llamadas.** Cada llamada a `predict()` arma y factoriza Σ de nuevo. En la validación cruzada eso significa una factorización por sitio y por modelo.
- **Validez del modelo.** Solo se detecta si la factorización de Cholesky falla.
- **Solo kriging simple.** No hay restricción de insesgamiento, de modo que los modelos sin meseta no se pueden usar.
- **Distancias euclidianas** sobre las coordenadas tal como vienen.